

Projektanleitung

Banana Supply Chain Datenplattform – Schritt-für-Schritt

Modul:	Datenmanagement und Analytics (M.Sc.), SoSe 26 – TH Lübeck
Deadline:	01.07.2026
Stand:	2026-05-13
Zweck:	Erklärt das gesamte Projekt: Aufgabe, Dateien, Datenbanken, Datenfluss und nächste Schritte

Fuer wen ist diese Anleitung? Fuer alle Projektmitglieder, die verstehen wollen, was das Projekt macht, wie die Dateien zusammenhaengen und wie man das System starten, befuellen und pruefen kann.

Inhaltsverzeichnis

1	Was ist das Projekt?	Aufgabenstellung, Use Case, Ziel
2	Wie ist das Projekt aufgebaut?	Ordnerstruktur, alle Dateien erklart
3	Die Infrastruktur (Docker)	Welche Datenbanken laufen, Ports, Passwörter
4	Datenfluss – von A nach B	Wie Daten erzeugt, verarbeitet und gespeichert werden
5	Die 5 Datenbanken im Detail	PostgreSQL, MongoDB, Redis, Neo4j, MinIO
6	Teil 1 – Was wurde erledigt?	Alle Artefakte und ihr Status
7	Wie starte ich das Projekt?	Schritt-fuer-Schritt-Anleitung zum Ausfuehren
8	Teil 2 – Was kommt noch?	Analytics-Aufgaben und naechste Schritte
9	Haeufige Fehler und Loesungen	Bekannte Probleme mit Workaround

1. Was ist das Projekt?

Das Projekt ist eine Studienleistung im Modul **Datenmanagement und Analytics (M.Sc.)**. Ziel ist es, eine vollstaendige Datenplattform fuer eine **Banana Supply Chain** zu entwerfen, zu implementieren und zu dokumentieren.

1.1 Der Use Case – Banana Supply Chain

Eine Bananenfarm in Ghana (Lieferant) verkauft Bananen an europaeische Supermaerkte (Kunden). Zwischen Ernte und Verkauf durchlaufen die Bananen mehrere Stationen:

Station	Beschreibung	System
Plantage (Ghana)	Ernte, Batches bilden, Qualitaetskontrolle	ERP
Sammelzentrum	Bananen von Plantagen buendeln	WMS
Qualitaetskontrolle	Temperatur und Qualitaet pruefen	WMS
Afrika Kuehlhaus	Tiefkuehlung vor Seetransport	WMS
Seetransport (Container)	GPS-Tracking, Temperaturueberwachung	TMS
Europa Kuehlhaus	Ankunft Europa, Zwischenlager	WMS
Zentrallager (DE)	Verteilung auf Regionen	WMS
Retail Store	Supermarkt, Abgabe an Kunden	ERP + TMS

1.2 Die Aufgabenstellung (Zusammenfassung)

Teil 1 – Datenmanagement (aktuell): Infrastruktur aufsetzen, Daten klassifizieren, Datenbanken modellieren, ETL implementieren, Qualitaet sichern. **Teil 2 – Analytics (noch offen):** KPIs berechnen, Python-Charts erstellen, PowerBI-Dashboard, Clustering, Absatzprognose.

Die vollstaendige Aufgabenstellung steht in **Aufgabenstellung.pdf** im Projektroot. Diese Datei ist die primaere Quelle – bei Unklarheiten immer dort nachschauen.

2. Wie ist das Projekt aufgebaut?

Der gesamte Projektordner liegt unter **gruppe7_dma_sose26/**. Hier ist eine Uebersicht aller wichtigen Ordner und Dateien:

2.1 Hauptordner auf einen Blick

Ordner / Datei	Was steckt darin?
bananasupplychain/	Das Herzstück: Docker-Setup, ETL-Skript, Datengenerator
shared/erp/	50 JSON-Dateien vom ERP-System (Lieferanten, Kunden, Produkte, Bestellungen)
shared/wms/	70 JSON-Dateien vom WMS-System (Lager, Stationen)
shared/tms/	263 JSON-Dateien vom TMS-System (Carrier, Transporte, GPS, Lieferungen)
docs/	13 Markdown-Dokumente – die gesamte fachliche Dokumentation
sql/	8 SQL-Dateien – PostgreSQL-Schemas, Tabellen, DWH, Qualitätschecks
cypher/	1 Cypher-Datei – das Neo4j-Graphmodell
Aufgabenstellung.pdf	Die originale Pruefungsaufgabe – immer zuerst lesen
PROJECT_STATUS.md	Aktueller Projektstatus (was ist fertig, was fehlt)

2.2 Alle wichtigen Dateien erklart

Python-Skripte:

Datei / Ordner	Was ist das?	Status
test_data_generator.py	Erzeugt die JSON-Dateien in shared/. Muss einmalig ausgefuehrt werden. Simuliert ERP, WMS und TMS.	getestet
etl_load.py	Liest die JSON-Dateien und laedt sie in alle 5 Datenbanken. Das Kernstueck des Projekts.	getestet
generate_documents.py	Erstellt PDF-Dokumente (Lieferscheine, Rechnungen) in MinIO und speichert Referenzen in PostgreSQL.	getestet
reorgFolders.py	Hilfsskript zum Reorganisieren der Ordnerstruktur.	erstellt

SQL-Dateien (Reihenfolge beachten!):

Datei / Ordner	Was ist das?	Status
sql/01_create_schemas.sql	Erstellt die 6 PostgreSQL-Schemas: erp, wms, tms, mdm, meta, dwh	getestet
sql/02_create_erp_tables.sql	Erstellt die 6 ERP-Tabellen (suppliers, customers, products, orders, orders_batches, batches)	getestet
sql/03_create_wms_tables.sql	Erstellt die 3 WMS-Tabellen + 7 Supply-Chain-Knoten als Stammdaten	getestet
sql/04_create_tms_tables.sql	Erstellt die 6 TMS-Tabellen (carriers, shipments, positions usw.)	getestet
sql/05_create_mdm_tables.sql	Erstellt das MDM-System (Golden Records, Source Mappings, Hilfsfunktionen)	getestet
sql/06_create_metadata_tables.sql	Erstellt den Metadaten-Katalog (Skalenniveaus, Qualitätsregeln)	getestet
sql/07_create_dwh_schema.sql	Erstellt das Sternschema fuer Analytics (7 Dimensionen + 1 Faktentabelle)	getestet
sql/08_data_quality_checks.sql	20+ SQL-Abfragen zur Datenpruefung (Vollstaendigkeit, Konsistenz usw.)	erstellt

Dokumentation (docs/):

Datei / Ordner	Was ist das?	Status
docs/01_data_classification.md	Klassifikation aller 13 Eventtypen (Stamm-/Bewegungs-/Ereignisdaten)	abgabefaeig
docs/02_target_architecture.md	Architekturdiagramm der gesamten Plattform	abgabefaeig
docs/03_er_model.md	Entity-Relationship-Modell mit Mermaid (PKs, FKs, Kardinalitaeten)	abgabefaeig
docs/04_masterdata_management.md	MDM-Konzept: wie BAN-101/BAN_101/ban-101 harmonisiert werden	abgabefaeig
docs/05_metadata_management.md	Metadaten-Konzept: Skalenniveaus fuer alle Kernspalten	abgabefaeig
docs/06_data_quality.md	6 Datenqualitaets-Dimensionen mit konkreten Regeln	abgabefaeig
docs/07_dwh_model.md	Data-Warehouse-Modell: Sternschema, Measures, ETL-Konzept	abgabefaeig
docs/08_mongodb_event_model.md	MongoDB: 4 Collections fuer Event-Daten	abgabefaeig
docs/09_redis_realtime_model.md	Redis: Key-Struktur fuer Echtzeit-Tracking	abgabefaeig
docs/10_neo4j_graph_model.md	Neo4j: Graphmodell mit 8 Node-Typen und 12 Beziehungen	abgabefaeig
docs/11_minio_document_model.md	MinIO: 4 Buckets fuer Lieferscheine, Rechnungen, Dokumente	abgabefaeig
docs/12_etl_concept.md	ETL-Konzept: Mapping-Tabelle fuer alle 13 Eventtypen	abgabefaeig

3. Die Infrastruktur (Docker)

Das Projekt laeuft vollstaendig in Docker-Containern. Die Konfiguration liegt in **bananasupplychain/container/docker-compose.yml**. Mit einem einzigen Befehl starten alle 5 Datenbanken:

```
cd bananasupplychain/container docker-compose up -d
```

3.1 Die laufenden Dienste

PostgreSQL (Port 5432)

Zweck: Relationale Datenbank fuer strukturierte Stamm- und Bewegungsdaten

Wofür: ERP-Tabellen (suppliers, orders), WMS, TMS, MDM, Metadaten, DWH

Beispiel: `SELECT * FROM erp.suppliers;` → zeigt alle Lieferanten

MongoDB (Port 27017)

Zweck: Dokumentendatenbank fuer Events mit flexibler Struktur

Wofür: Shipment-Events, GPS-Updates, Node-Events, Batch-Tracking

Beispiel: `db.shipment_events.find({shipment_id: 'SHIP-001'})`

Redis (Port 6379)

Zweck: In-Memory-Cache fuer Echtzeit-Daten mit sub-Millisekunden-Latenz

Wofür: Aktueller Shipment-Status, GPS-Position, aktive Transporte zaehlen

Beispiel: `GET shipment:SHIP-001:status` → 'IN_TRANSIT'

Neo4j (Port 7474 (Browser) / 7687 (API))

Zweck: Graphdatenbank fuer Beziehungsnetzwerke und Pfadabfragen

Wofür: Supply-Chain-Routen, wer liefert an wen, kuerzeste Pfade

Beispiel: `MATCH p=(s:Supplier)-[*]->(c:Customer) RETURN p`

MinIO (Port 9000 (API) / 9001 (Web))

Zweck: S3-kompatibler Object-Store fuer Binaerdokumente

Wofür: PDFs: Lieferscheine, Rechnungen, Transportdokumente, Batch-Zertifikate

Beispiel: Browser: `http://localhost:9001` (admin / password)

Zugangsdaten (lokal): User: user | Password: password | Database: logistics (PostgreSQL) | Neo4j: neo4j / password | MinIO: admin / password

4. Datenfluss – von A nach B

Hier erklärt, wie Daten von der Simulation bis in die Datenbanken fließen – in vier klar getrennten Phasen:

1	Datengenerator (Simulation)	test_data_generator.py läuft einmalig und erzeugt 383 JSON-Dateien in shared/. Er simuliert drei Quellsysteme: ERP (50 Dateien), WMS (70 Dateien), TMS (263 Dateien). Absichtlich eingebaute Inkonsistenz: Produktcode BAN-101 heisst in WMS 'BAN_101' und in TMS 'ban-101'.
2	Extract – JSON lesen und validieren	etl_load.py liest alle JSON-Dateien mit glob.glob(). Jede Datei wird geöffnet, der Eventtyp identifiziert (z.B. 'SupplierCreated', 'ShipmentStarted') und die Pflichtfelder validiert. Ungültige Dateien werden uebersprungen.
3	Transform – Harmonisieren und konvertieren	Schlüsselproblem: BAN_101 (WMS) muss zu BAN-101 (kanonisch) werden. Die Funktion normalize_key() macht: strip().lower().replace('_', '-').upper(). Ausserdem: Strings zu Zahlen konvertieren, Timestamps vereinheitlichen, fehlende Felder mit None befüllen.
4	Load – In alle 5 Systeme laden	Jedes Event geht in das passende Zielsystem: Stammdaten (Supplier, Customer) → PostgreSQL erp-Schema. GPS-Events → MongoDB shipment_events + Redis Echtzeit-Key. Alle Shipments → Neo4j als Graph-Knoten. Dokumente → MinIO mit Referenz in PostgreSQL erp.document_references.

4.1 Welches Event geht wohin?

Eventtyp	Quellsystem	Primaer-Ziel	Sekundaer-Ziel
SupplierCreated	ERP	PostgreSQL erp.suppliers	Neo4j Supplier-Node
CustomerCreated	ERP	PostgreSQL erp.customers	Neo4j Customer-Node
ProductCreated	ERP	PostgreSQL erp.products	MDM Golden Record
OrderCreated	ERP	PostgreSQL erp.orders	MongoDB order_events
BatchCreated	ERP	PostgreSQL erp.batches	Neo4j Batch-Node
WarehouseSKUCreated	WMS	PostgreSQL wms.warehouse_skus	MDM Mapping (BAN_101)
NodeProcessed	WMS	PostgreSQL wms.node_processings	MongoDB node_events
CarrierCreated	TMS	PostgreSQL tms.carriers	Neo4j Carrier-Node
TransportProductRefCreated	TMS	PostgreSQL tms.transport_product_refs	MDM Mapping (ban-101)
ShipmentStarted	TMS	PostgreSQL tms.shipments	MongoDB shipment_events, Neo4j
ShipmentPositionUpdated	TMS	Redis (Echtzeit)	MongoDB shipment_events
TransportCompleted	TMS	PostgreSQL tms.completions	MongoDB update
DeliveryCompleted	TMS	PostgreSQL tms.deliveries	MinIO Lieferschein

5. Die 5 Datenbanken im Detail

5.1 PostgreSQL – der operative Kern

PostgreSQL ist die zentrale relationale Datenbank. Sie enthaelt 6 Schemas, 26 Tabellen und eine DWH-Schicht. Reihenfolge beim Ausfuehren wichtig:

Schema	Enthaeht	Tabellen
erp	Enterprise Resource Planning – Stammdaten und Bestellungen	suppliers, customers, products, orders, order_items, batches, document_references
wms	Warehouse Management – Lager und Stationsverwaltung	warehouse_skus, supply_chain_nodes (7 Knoten vorbefuehrt), node_processings
tms	Transport Management – Carrier und Transporte	carriers, transport_product_refs, shipments, positions, completions, deliveries
mdm	Master Data Management – Schluesselharmonisierung	entitytypes, golden_records, source_mappings + Funktion resolve_canonical_key()
meta	Metadaten-Katalog – Skalenniveaus und Qualitätsregeln	tables, columns (mit quality_rule und scale_level)
dwh	Data Warehouse – Sternschema fuer Analytics	dim_date (1095 Zeilen), dim_supplier, dim_customer, dim_product, dim_carrier, dim_rou

5.2 MDM – das Schluesselproblem

Problem: Das Produkt 'Cavendish Bananen' hat in ERP den Code BAN-101, in WMS den Code BAN_101 und in TMS den Code ban-101. Ohne MDM koennen diese Systeme nicht miteinander kommunizieren – ein JOIN auf Produktebene wuerde nichts finden. **Loesung:** Die Funktion mdm.resolve_canonical_key('BAN_101', 'WMS') gibt immer BAN-101 (den kanonischen ERP-Schluessel) zurueck.

5.3 MongoDB – Events mit flexibler Struktur

MongoDB speichert Events als JSON-Dokumente. Der Vorteil: Jeder Eventtyp hat andere Felder – GPS-Events haben Koordinaten, Delivery-Events haben Empfaenger. In PostgreSQL muesste man fuer jeden Typ eine eigene Tabelle anlegen oder mit NULL-Spalten arbeiten. MongoDB loest das elegant durch flexible Schemas.

4 Collections: **shipment_events** (500 Dokumente), **node_events** (121), **batch_tracking** (11), **order_events** (11)

5.4 Redis – Echtzeit in Millisekunden

Redis speichert Daten im Arbeitsspeicher (RAM). Daher ist er 100x schneller als PostgreSQL fuer Lese-Anfragen. Typische Anfrage: 'Wo ist Shipment SHIP-001 gerade?' Redis antwortet in < 1 ms. Konfiguration: max 256 MB, aelteste Keys werden automatisch geloescht (allkeys-lru).

5.5 Neo4j – Beziehungen als Graph

Neo4j modelliert die Supply Chain als Netzwerk aus Knoten (Supplier, Customer, Carrier, Batch etc.) und Beziehungen (SUPPLIES, TRANSPORTS, PROCESSED_AT). Die Frage 'Welchen Weg hat Batch B-001 durch die Supply Chain genommen?' beantwortet Neo4j mit einer einzigen Cypher-Zeile. In SQL brauchte man 6+ JOINS.

Ergebnis nach ETL: 125 Nodes, 47+ Relationships. Pfad PLANTATION -> RETAIL: 6 Hops.

5.6 MinIO – Dokumente ausserhalb der Datenbank

PDFs (Lieferscheine, Rechnungen) gehoeren nicht in eine relationale Datenbank – das wuerde Backups und Replikation verlangsamen. MinIO ist ein S3-kompatibler Object Store: Die PDFs liegen in Buckets, PostgreSQL speichert nur den Verweis (Bucket + Dateiname). Web-Interface unter <http://localhost:9001>

6. Teil 1 – Was wurde erledigt?

Alle Pflichtanforderungen aus Teil 1 der Aufgabenstellung sind erfüllt. 35 von 35 Anforderungen haben Status abgabefähig oder getestet.

Infrastruktur

- ✓ Docker: 5 Container laufen (PostgreSQL, MongoDB, Redis, Neo4j, MinIO)
- ✓ Datengenerator: 383 JSON-Events erzeugt (ERP 50, WMS 70, TMS 263)
- ✓ GitLab-Repo eingerichtet und befüllt

Datenmodelle

- ✓ PostgreSQL: 26 Tabellen in 6 Schemas
- ✓ ER-Modell mit PKs, FKs, Kardinalitäten (Mermaid)
- ✓ MDM: Golden Records für Produkte, Carrier, Supplier

Spezial-Systeme

- ✓ MongoDB: 4 Collections, 643 Dokumente
- ✓ Redis: STRING, HASH, SORTED SET, COUNTER – alle getestet
- ✓ Neo4j: 125 Nodes, Pfad PLANTATION->RETAIL in 6 Hops
- ✓ MinIO: 4 Buckets, 66 Dokument-Referenzen

ETL & Qualität

- ✓ etl_load.py: 383 Events vollständig geladen
- ✓ 20+ DQ-Checks für 6 Qualitäts-Dimensionen
- ✓ Metadaten-Katalog mit Skalenniveaus für alle Kernspalten

7. Wie starte ich das Projekt?

Vollstaendige Anleitung zum Ausfuehren aller Projektkomponenten auf einem neuen Rechner:

1	Voraussetzungen pruefen	Docker Desktop muss installiert und gestartet sein. Python 3.10+ muss installiert sein. Befehl: <code>docker --version && python3 --version</code>
2	Abhaengigkeiten installieren	<code>pip install psycpg2-binary pymongo redis neo4j minio</code> Diese Pakete benoetigt <code>etl_load.py</code> fuer alle Datenbankverbindungen.
3	Docker-Container starten	<code>cd bananasupplychain/container docker-compose up -d</code> Warten bis alle 5 Container den Status 'healthy' haben. Pruefen: <code>docker ps</code>
4	SQL-Schemas anlegen (Reihenfolge 01 bis 08)	In psql oder einem SQL-Tool (z.B. DBeaver) verbinden: Host: localhost Port: 5432 DB: logistics User: user PW: password Dann ausfuehren: <code>sql/01...</code> bis <code>sql/08...</code> in dieser Reihenfolge.
5	Datengenerator ausfuehren (falls shared/ noch leer)	<code>cd bananasupplychain python3 test_data_generator.py</code> Erzeugt: 50 ERP + 70 WMS + 263 TMS JSON-Dateien in shared/
6	ETL-Skript ausfuehren	<code>cd bananasupplychain python3 etl_load.py</code> Laedt alle 383 Events in PostgreSQL, MongoDB, Redis, Neo4j und MinIO. Erwartete Ausgabe: Zusammenfassung mit Anzahl geladener Datensatze pro System.
7	Dokumente generieren (MinIO befuellen)	<code>python3 generate_documents.py</code> Erstellt 60 Lieferscheine + 6 Rechnungen als PDFs in MinIO. Speichert 66 Referenzen in PostgreSQL <code>erp.document_references</code> .
8	Ergebnis pruefen	PostgreSQL: <code>SELECT COUNT(*) FROM erp.suppliers;</code> -- erwartet: 10 MongoDB: <code>db.shipment_events.countDocuments()</code> -- erwartet: 500 Redis: <code>KEYS shipment:*</code> -- zeigt alle Shipment-Keys Neo4j Browser: <code>http://localhost:7474</code> -- MATCH (n) RETURN COUNT(n) MinIO: <code>http://localhost:9001</code> -- Buckets mit Dokumenten sehen

8. Teil 2 – Was kommt noch?

Teil 2 (Analytics) ist vollstaendig offen. Deadline: 01.07.2026. Voraussetzung: ETL Phase 2 muss zuerst implementiert werden (PostgreSQL -> DWH befuellen).

#	Aufgabe	Beschreibung	Priori.
A-0	ETL Phase 2 (Blocker)	Python-Skript: operative Schemas -> DWH-Schema befuellen. Ohne das koennen A-1 bis A-6 nicht starten	Hoch
A-1	Deskriptive Statistik	Min, Max, Mittelwert, Median, Std fuer delay_minutes, temperature, quantity	High Python + pandas.
A-2	KPI-Berechnung	5 KPIs: Liefertreue (%), Transportdauer (Tage), Temperatúrausreisser-Count, Bestellwert, Batchqualität	Hoch
A-3	5 Python-Charts	Matplotlib/Seaborn: Histogramm, Zeitreihe, Boxplot, Heatmap, Liniendiagramm. Je Chart: fachliche Interpretation	Hoch
A-4	PowerBI-Dashboard	Verbindung zu PostgreSQL DWH. KPI-Cards, Zeitreihe, Slicer fuer Datum/Carrier/Route.	Hoch
A-5	Clustering	k-Means auf Kunden (Bestellhaeufigkeit, Bestellwert, Verzoegerung). Elbow-Methode fuer k.	Mittel
A-6	Absatzprognose	ARIMA oder Prophet: Bestellvolumen pro Woche prognostizieren. RMSE/MSE angeben.	Mittel
A-7	Abschlussbericht	Zusammenfassung Teil 1 + Teil 2 in einem Dokument.	Hoch

9. Haeufige Fehler und Loesungen

F-1: git: Unable to create index.lock

Ursache: VS Code und Terminal versuchen gleichzeitig auf git zuzugreifen.

Loesung: `rm .git/index.lock` Danach `normal` committen.

F-2: etl_load.py: Duplicate key violation in PostgreSQL

Ursache: Das ETL-Skript wurde mehrfach ausgefuehrt. `ON CONFLICT DO NOTHING` fehlt noch.

Loesung: Datenbank zuruecksetzen: `psql -h localhost -U user -d logistics -c 'DROP SCHEMA erp CASCADE;'` Dann SQL 01-08 neu ausfuehren, danach `etl_load.py` einmal laufen lassen.

F-3: pip install schlaegt fehl (psycopg2)

Ursache: Auf macOS fehlen oft PostgreSQL-Bibliotheken.

Loesung: `pip install psycopg2-binary` (nicht `psycopg2`!)

F-4: Neo4j: ServiceUnavailable

Ursache: Neo4j braucht nach `docker-compose up` ca. 30-60 Sekunden zum Starten.

Loesung: `docker logs neo4j --` auf `'Started.'` warten, dann `etl_load.py` nochmal starten.

F-5: MinIO: NoSuchBucket

Ursache: `generate_documents.py` wurde vor `etl_load.py` ausgefuehrt. ETL legt Buckets an.

Loesung: Reihenfolge: 1. `etl_load.py` 2. `generate_documents.py`